Decision Models as Universal Tool Selectors

Aziz Muminov Alexandra Ilina

November 2026

Abstract

Agents increasingly work through many tools: a coding agent connected to a company's MCP servers can face hundreds or thousands of tool definitions. Loading them all is expensive: for 613 tools the largest prompt reached about 182k tokens and a run cost 8–9 times as much as with search. Shipped harnesses therefore search instead, mostly with BM25, as in Codex, Anthropic's tool search and Hermes Agent. Lexical search is cheap but imprecise, and every extra search is a full agent turn. We propose decision models, which return a probability per option in one forward pass, as general-purpose tool selectors. Our selector, built on Microsoft Decision 1, chooses the service, then the tools, with a "none" option and usage-based recommendations, and needs about three decision-model calls per search. On the real tool definitions of 30 MCP servers with 1,076 tools it ranks far better than BM25 offline, with the right tool first for 92% of hand-written queries against 49%. On the ToolRet benchmark, 7,961 queries over 44,453 tools, the same decision model re-ranking candidates from BM25 and embeddings scored 53.6 NDCG@10, above every published retriever and re-ranker, including models trained for tool retrieval. Inside a shipped agent, Hermes Agent, on 37 coding tasks with simulated back ends, pre-planned paired runs made at the same time, five per task, used 9.7% fewer tokens with GPT-6.1 Sol, $p = 0.004$, and 8.5% fewer with DeepSeek V4.1 Flash, $p = 0.07$, with 32–60% fewer searches and equal pass rates. The selector's fee left the cost of a Sol run unchanged and made DeepSeek runs 25% more expensive. These results cover coding agents, simulated services, two agent models and one harness.

1 Introduction

Through the Model Context Protocol, MCP [19], an agent can read CI logs, error trackers, tickets, chat, dashboards, clusters and databases. Each server exposes tens to hundreds of tools, and agents connect to many servers, so which tools the agent sees, and how it finds the rest, now shapes cost and behaviour.

There are two options. The first is to load every definition into every request. For the 613 tools of twelve popular servers the largest prompt then reaches about 182k tokens, against 12–16k with search, and a run costs 8–9 times as much at the same pass rate (§6.4). The second, which harnesses ship, is search: definitions stay out of the prompt and the agent finds tools on demand (§2). Most shipped search is BM25 [2, 21, 23]. BM25 is cheap and needs no model, but it is imprecise, so agents search repeatedly, and each search is a full LLM turn that re-reads the whole context. With a frontier agent model, a wasted search costs far more than the search itself.

Decision models, or DMs, read a state and a question with named options and return a probability per option in one forward pass, without generating text. They are cheap and fast, and choosing among named options is what tool selection asks for. Our thesis is that a DM can serve as a drop-in, harness-agnostic tool selector: one API call per search, no training, and no change to the agent. We test it with Microsoft Decision 1 [17].

Contributions.

- A DM tool selector, selector F, and the analyses behind its design: choose services first with a "none" option, then ask two questions per service, and fill a fixed-size answer with usage-based names (§3, §6.5).
- A test suite built on the real tool definitions of 30 MCP servers, with simulated back ends, 38 graded coding tasks and isolated agent runs (§4), and a protocol for measuring token differences between agent configurations: same-time pairs, task-level analysis and written analysis plans (§5).
- Results for two agent models in Hermes Agent: selector F used 9.7% fewer tokens than BM25 with GPT-6.1 Sol and 8.5% fewer with DeepSeek V4.1 Flash, at equal pass rates (§6.1).
- An external check on the ToolRet benchmark: as a re-ranker, the DM scores above every published system with and without ToolRet's instructions (§6.7).

The scope is limited: coding agents, simulated services, two agent models and one harness (§8). "Universal" names the proposal, a selector that needs nothing from the harness but a query, not a result we have shown for every setting.

2 Background: how harnesses select tools, and decision models

Harnesses handle large catalogs in one of four ways. Cursor, Cline and opencode load every definition into every request. Codex [23], Anthropic's tool search [2] and Hermes Agent [21] search with Okapi BM25 [27] over names, descriptions and parameters. Claude Code's ToolSearch [3] lists the names of deferred tools and searches by keyword weighted towards names, and GitHub Copilot [16] loads groups of tools on demand. Embedding search [11] and LLM routers [9] have been tried, for example in Goose [6], which has since removed them. All but full loading give the agent a search tool and a way to call what it finds, so the agent pays for a definition only when it retrieves it.

Hermes Agent's bridge. Hermes Agent 0.19.0 defers MCP tools once their definitions exceed 10% of the context window, as hundreds of tools do. Its `tool_search` returns compact hits, a name and one line each; `tool_describe` returns one full definition; found tools are then called directly with typed arguments. Only the ranker behind `tool_search` differs between our conditions.

Decision models. A request to OpenRouter's Decisions API [25] carries a *state*, here the agent's search query, and one or more named questions. A multiple-choice question lists up to 255 options, each with a free-text description, and returns a probability per option; a yes/no question returns one probability. Microsoft Decision 1 is post-trained from Qwen3.5-9B [17] and costs about $0.04 per million input tokens. We also benchmarked TypeSafe Jev [30] offline at 1,076 tools: both put the right tool first for 87.5% of queries and in the top five for 95.8%; Jev answered twice as fast, a median of 0.27 against 0.57 s, but cost $0.211 against $0.145 per 1,000 searches and wrongly answered "none" for 1.7% of queries against 0%. We use Microsoft Decision 1 throughout.

3 The DM tool selector

Selector F answers each `tool_search` query in two rounds of DM calls (Figure 1).

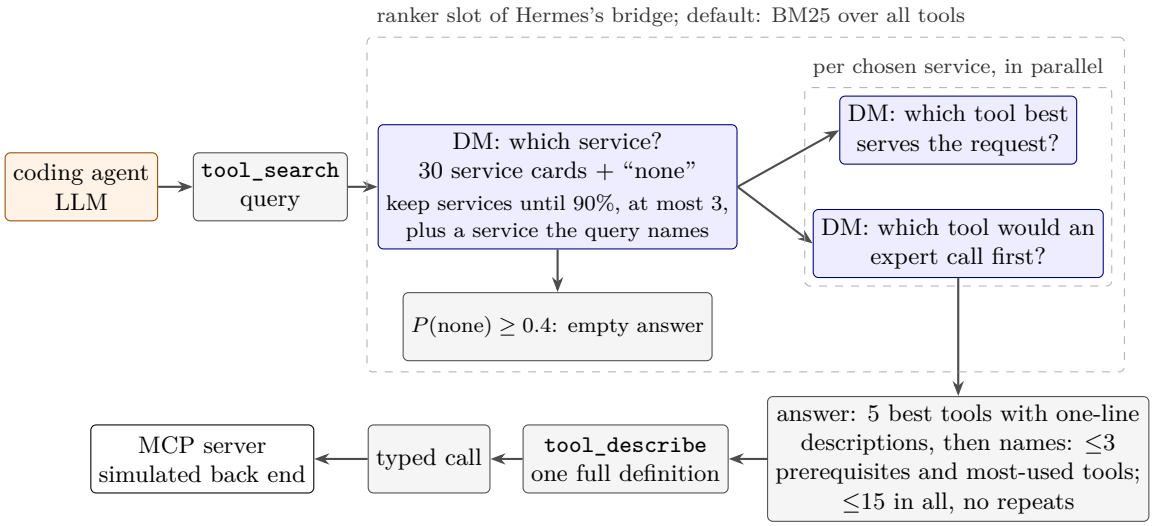


Figure 1: Selector F inside Hermes Agent's search bridge. The agent's query goes to the ranker slot. One DM call chooses the services or answers "none"; then two DM calls per chosen service run in parallel. The agent receives five compact hits and a list of names, reads full definitions with `tool_describe` and calls tools directly. In the baseline, BM25 fills the same slot; everything outside the slot is identical.

Service step. One multiple-choice question, "Which service has the tool the agent needs?", lists a card for each of the 30 services and a "none" option. If $P(\text{none}) \geq 0.4$ the answer is empty. Otherwise the selector takes services in order of probability until they cover 90% of the probability, at most three, and always includes a service that the query names. Card quality matters: the original cards confused GitHub, Git and GitLab, and the fixed cards say that GitHub hosts branches, commits and files, that Git is only the local working copy, and that GitLab hosts legacy projects (§6.2).

Tool step. For each chosen service, in parallel, two multiple-choice questions over its tools, each tool described by its name, the start of its description and its parameter names: "Which tool best serves what the agent wants to do?" and "Which tool would an expert engineer call first to do what the agent wants?". Scores are weighted by the service probability. The answer holds the five tools that best serve the query, with Hermes's one-line descriptions, and then, as names only, up to three prerequisites from the second question and the chosen services' most-used tools, at most 15 tools in all. Tools already shown in the run are not repeated. The usage lists are built leave-one-task-out from an earlier run, so a task never sees usage from its own runs.

Why this shape. Agents work service by service and use three to four tools per service, most of them in later steps; descriptions are expensive and names are cheap; and the DM's probabilities are too peaked to size a list, so the list size is fixed and filled by prerequisites and usage. The offline study of §6.5 chose each element.

Cost. A search takes about three DM calls in two sequential rounds. In the agent runs a search took a median of 1.1 s and cost $0.35 per 1,000 searches; 5.1% of searches got the empty "none" answer, all of them with DeepSeek.

4 Experimental setup

Tool catalog. We started each MCP server as an unprivileged user with dummy credentials and saved its `tools/list` response: names, descriptions and input schemas exactly as published. No tool was ever executed against a real service. The main catalog has 613 tools from 12 popular servers; 18 further servers bring it to 1,076 tools, including decoys such as GitLab beside GitHub, AWS CloudWatch beside Grafana and AWS EKS beside Kubernetes. App. B lists all servers.

Simulated back ends. Each task has a world of rules that map a tool and matching argument values to a response, followed by organisation-wide facts shared by all tasks. Unmatched reads return an empty list or "not found"; unmatched writes succeed and are recorded for grading. Arguments are validated against the tool's schema.

Tasks. Each task is a small Python repository whose fix needs information that exists only in the services, such as a failing CI check, a Sentry stack trace or a decision in Slack. Task set v2 has 38 *symptom-first* tasks that do not name the service: 23 single-service, 7 multi-service, 4 *absent*, where the information does not exist and the right outcome is to stop and leave a `TODO`, and 4 controls that need no catalog tool (App. B). Every repository carries the same short handbook of where things live, like an `AGENTS.md`, and every prompt starts with the same note that the company's services are connected as tools, some loaded on demand through tool search, and ends with the same escape clause: if the information is not available, do not guess. One task, `k8s_crashloop`, is excluded from the agent runs for both conditions, because its grading rule forbids a reasonable read-only `kubectl exec`. The offline queries and the opencode runs use the earlier set v1, 33 tasks whose prompts often name the service.

Grading. A run passes when the hidden tests pass, every required write was made with the right arguments, and no destructive call such as a merge, delete or rollout restart was made. We checked that each task's hidden tests fail on the original repository and pass with the reference solution, and that the reference calls get the intended answers.

Harnesses and isolation. The agent runs use Hermes Agent [21] 0.19.0 with its own tool-search bridge over all 1,076 tools; we patched its search function to ask our server's ranking endpoint, so only the ranker differs. Supporting runs use opencode [24] 1.19.4. Agents run as an unprivileged user without credentials, each Hermes run in its own mount namespace, with a per-run token that the agent's shell cannot read and a deny list of probing commands (App. A).

Agent models and baseline. DeepSeek V4.1 Flash [8] is pinned to DeepInfra's fp8 endpoint with temperature 0.6; GPT-6.1 Sol is pinned to OpenAI with default sampling. The baseline is Hermes Agent's built-in BM25, unmodified. Each task ran five times under each condition with each model, 185 same-time pairs per model; for DeepSeek the five repeats per task were drawn at random from 15 with a seed fixed before the analysis. *Tokens* are all uncached input, cached input and output tokens summed over a run's LLM calls; *cost* is the LLM cost at the pinned provider's prices, including cache writes where billed, plus the DM fee.

5 Measuring token differences in agents

Pass rates are near ceiling for both rankers, so the differences we look for are in tokens, and tokens are noisy.

Run-to-run noise. Two runs of the same task under the same condition can differ by a large factor. The standard deviation of the per-slot log ratio between conditions was 0.58 with DeepSeek, so a single slot easily swings by a factor of 1.8, and 0.29 with Sol (Figure 2).

Drift. Batches run at different times are not directly comparable. In an A/A comparison, BM25 against BM25, two batches run some days apart differed by 0.94× [0.87, 1.02] over 111 matched slots of 37 tasks, $p = 0.16$. That is not significant, but the interval admits differences as large as the effects we measure.

Protocol. Both conditions run the same task and repeat number back to back, a *slot*. For each slot we take $\log(\text{tokens}_F/\text{tokens}_{\text{BM25}})$, average it per task, and test the 37 task means with a two-sided one-sample t-test; the 95% interval is a task-level bootstrap with 10,000 resamples, reported as a geometric-mean ratio. A verdict of "fewer tokens" needs $p < 0.05$ and an interval that excludes 1. Treating slots as independent would overstate the evidence, since repeats of a task are correlated. Before each batch we wrote an analysis plan, logged every interim look, and ran the batch to completion regardless of interim numbers; the plans are released with the run records. Runs that ended in provider errors without recorded tokens were rerun; nothing else was excluded.

Power. Simulations from the measured variance give five repeats about 98% power to detect a 12.5% token reduction at Sol's noise, but only about 80% at DeepSeek's.

6 Results

6.1 Tokens inside Hermes Agent

Each of the 37 tasks ran five times under BM25 and under selector F with each agent model, 185 same-time pairs per model. Selector F used fewer tokens with both models (Table 1, Figure 2). With GPT-6.1 Sol the reduction was 9.7%, 0.903× [0.84, 0.96], $p = 0.004$: Sol searched 2.2 times per run instead of 5.5, 60% fewer, and made 12% fewer LLM calls. With DeepSeek V4.1 Flash it was 8.5%, 0.915× [0.84, 1.01], $p = 0.07$, with 32% fewer searches, 7.3 against 10.7. DeepSeek's runs are noisier, so at five repeats its interval just reaches 1. Pass rates were equal: 173 against 172 of 185 with Sol and 182 against 184 with DeepSeek, McNemar $p = 1.0$ and 0.63. Wall time did not change. The saving was largest on the 7 multi-service tasks, 0.76× with Sol and 0.80× with DeepSeek, against 0.94× and 0.91× on the 22 single-service tasks.

6.2 Service errors: the Git family

We asked the DM's service question for 2,477 distinct agent queries that name exactly one service. It chose the named service for 97.9%. Of its 51 errors, 47 were within Git, GitHub and GitLab, most often Git for a GitHub query, and in 45 the named service came second. By service it chose correctly 608 of 639 times for GitHub, 86 of 95 for Git and 325 of 332 for GitLab, and in every case for Atlassian, Sentry, Slack, Notion, CloudWatch, MongoDB, Postgres and EKS. The errors came from the original service cards, which did not say that pull requests and branches live on GitHub rather

Table 1: Selector F against BM25 in Hermes Agent: 37 tasks, 5 repeats, 185 same-time pairs per model. Per-run metrics as mean / median. Ratio: geometric mean of per-slot ratios of selector F to BM25, averaged per task, with a task-level bootstrap 95% CI; p: t-test over the 37 task means; for searches, the paired difference per run. Passed runs: runs passed only by BM25 against only by selector F, McNemar test. Cost includes cache writes and the DM fee. Bold: the better value where the difference is significant.

Model	Metric	BM25	selector F	selector F vs BM25 [95% CI]	p
GPT-6.1 Sol	Passed runs	172/185	173/185	4 vs 5	1.00
	Tokens per run	193k / 168k	**171k / 133k**	0.903× [0.84, 0.96]	0.004
	LLM calls per run	13.4 / 13	**11.5 / 10**	0.88× [0.83, 0.92]	<0.0001
	Searches per run	5.5 / 5	**2.2 / 2**	−3.3 [−4.4, −2.2]	<0.0001
	Cost per run, $	0.0724 / 0.0641	0.0715 / 0.0630	0.99× [0.95, 1.03]	0.63
	Wall time per run, s	53 / 45	52 / 41	0.99× [0.92, 1.05]	0.69
DeepSeek V4.1 Flash	Passed runs	184/185	182/185	3 vs 1	0.63
	Tokens per run	769k / 552k	717k / 477k	0.915× [0.84, 1.01]	0.07
	LLM calls per run	28.5 / 26	27.4 / 24	0.95× [0.89, 1.01]	0.07
	Searches per run	10.7 / 10	**7.3 / 6**	−3.4 [−4.5, −2.3]	<0.0001
	Cost per run, $	**0.0080 / 0.0067**	0.0104 / 0.0084	1.25× [1.15, 1.36]	<0.0001
	Wall time per run, s	135 / 123	145 / 128	1.04× [0.95, 1.14]	0.38

than in the local Git copy; selector F therefore uses fixed cards for the three services and always includes a service the query names, and it passed `gh_open_pr` in all 10 of its runs. The remaining failures are hard for both rankers: with Sol, `loki_errors` and `prom_threshold` account for 8 of BM25's 13 failures and 10 of selector F's 12; with DeepSeek, selector F failed `prom_threshold` twice and BM25 failed one run.

6.3 Cost

The DM fee is fixed per search, while any saving is a share of the agent's own cost. With Sol the fee was $0.0008 per run against an LLM cost of about seven cents, and cost per run did not change, 0.99× [0.95, 1.03]: more than half of Sol's cost was cache writes, which selector F does not reduce. With DeepSeek the fee was $0.0026 per run, about a third of a run's cost, and runs became 25% more expensive, 1.25× [1.15, 1.36]; before the fee, selector F's runs cost 2% less. The tokens saved are mostly cached input, the cheapest kind, so a fee per search pays off only where searches are frequent or the agent model's uncached input and output are expensive.

6.4 Loading all tools: opencode

Supporting runs with opencode on the 33 v1 tasks and 613 tools, two repeats per task and two inexpensive agent models, show why harnesses search (Table 2). Without a catalog no tool task passed; with one, every condition passed 96–100% of runs. Loading every definition made the largest prompt about 182k tokens with DeepSeek; with ranked search it was 12–16k, and the cost per run fell from $0.072 to $0.008–0.009, 8–9 times less. Pass rate cannot separate the search mechanisms, which is why the Hermes runs compare tokens.

6.5 Designing the selector

We compared 13 selector variants on three fixed query sets at 1,076 tools: 120 hand-written queries for the steps of task set v1, plus 40 that no tool can serve; 400 searches made by agents in BM25 runs of an earlier batch, labelled with the new tools the agent used after each search and before its

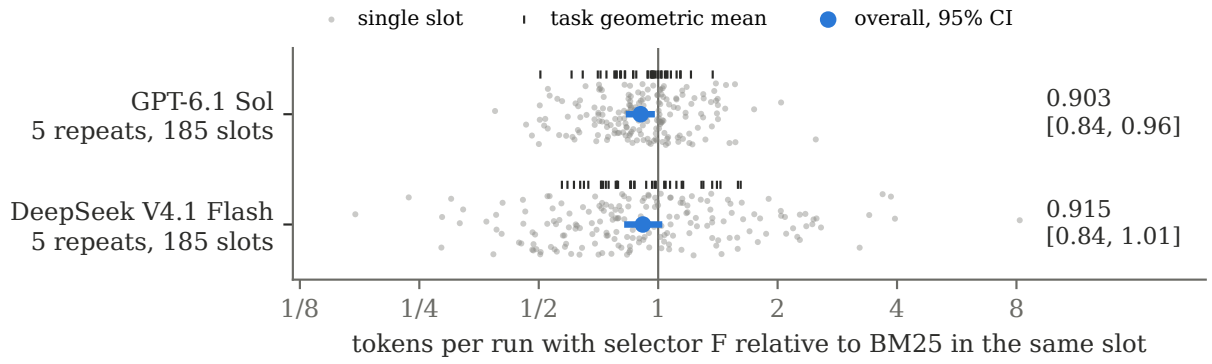


Figure 2: Tokens per run with selector F relative to BM25 in the same slot. Grey: single slots; black ticks: the 37 task-level geometric means; blue: overall geometric mean of the task means with a task-level bootstrap 95% CI. With DeepSeek, single slots range from 0.17 to 8.2, so only paired averages over many slots are informative.

Table 2: opencode on task set v1, 613 tools, two repeats per task. Pass: runs passed on the 25 tasks that need a catalog tool. Largest prompt: median over those runs. Cost: mean LLM cost per run over all 33 tasks, without the selector's fee. Bold: smallest largest prompt and lowest cost among conditions with catalog access.

	DeepSeek V4.1 Flash			Tencent hy4		
Condition	Pass	Largest prompt	$/run	Pass	Largest prompt	$/run
no catalog	0/25	9.6k	0.0110	0/25	15.4k	0.0154
all tools loaded	25/25	181.9k	0.0722	25/25	188.5k	0.1039
Claude Code style	25/25	16.2k	0.0089	25/25	**32.0k**	**0.0375**
BM25	25/25	13.4k	0.0093	25/25	33.6k	0.0417
tool groups	25/25	29.0k	0.0179	24/25	52.7k	0.0562
embeddings	25/25	13.3k	0.0090	23/25	49.0k	0.0514
LLM router	25/25	**11.7k**	**0.0076**	25/25	32.6k	0.0419
DM hierarchy	25/25	14.0k	0.0082	25/25	34.6k	0.0470

next one; and 350 agent queries that name exactly one service, 200 of them Git, GitHub or GitLab. Table 3 gives the results.

Cards and the service nucleus matter most. Taking services until 90% of the probability instead of only the top one raised hit@5 from 0.90 to 0.96, and fixing the GitHub, Git and GitLab cards raised hit@1 from 0.88 to 0.92. A rule that always includes a named service put it first on all 350 named queries.

The "none" option is free. It answered "none" for 95% of the 40 queries that no tool serves, never for a query that one does, and left every other column unchanged.

Wording matters little; other shapes are worse or costlier. Rephrasing the questions moved hit@1 by a few points at most. Asking yes or no per tool lowered hit@1 to 0.75 within services and to 0.63 over all tools, where it also lost the named service a third of the time. A sliding window over all tools, without a service step, covered slightly more next tools but cost about 12

Table 3: Offline selector study at 1,076 tools with Microsoft Decision 1. Hand: hit@k on 120 hand-written queries. Agent: share of the next tools of 400 agent searches covered by the first 5 or all 15 tools, and share of searches with all next tools covered. Named: the named service ranked first, on 350 queries and on the 200 in the Git family. Cost: DM fee per 1,000 searches. Variants 4–13 build on variant 3. Bold: best per column.

	Hand			Agent			Named		
Variant	hit@1	hit@5	hit@15	first 5	all 15	all covered	all	Git family	$/1k
1 top service only	0.87	0.90	0.91	0.80	0.83	0.75	0.97	0.95	0.18
2 services until 90%	0.88	0.96	0.96	0.83	0.85	0.78	0.96	0.94	0.20
3 2 + fixed cards	**0.92**	0.98	0.99	0.83	0.86	0.79	0.98	0.97	0.19
4 “choose the named service”	**0.92**	0.98	0.99	0.83	0.86	0.79	0.98	0.97	0.20
5 “none” option	**0.92**	0.98	0.99	0.83	0.86	0.78	0.98	0.97	0.19
6 named-service rule	**0.92**	0.98	0.99	0.83	0.86	0.79	**1.00**	**1.00**	0.19
7 “expert first” question	0.83	**0.99**	**1.00**	0.83	0.85	0.78	0.98	0.98	0.20
8 yes/no per tool	0.75	0.97	**1.00**	0.82	0.86	0.80	0.98	0.97	0.29
9 sliding window, no service step	0.86	**0.99**	**1.00**	**0.84**	0.86	**0.81**	0.98	0.99	2.26
10 yes/no on all tools, no service step	0.63	0.86	0.95	0.73	0.83	0.76	0.67	0.67	3.43
11 “which tools are relevant”	0.88	0.98	0.99	0.83	0.86	0.79	0.98	0.97	0.20
12 best serves and expert first	**0.92**	0.98	**1.00**	0.83	0.86	0.79	0.98	0.97	0.36
13 best serves and relevant	**0.92**	0.98	0.99	0.83	0.86	0.79	0.98	0.97	0.36

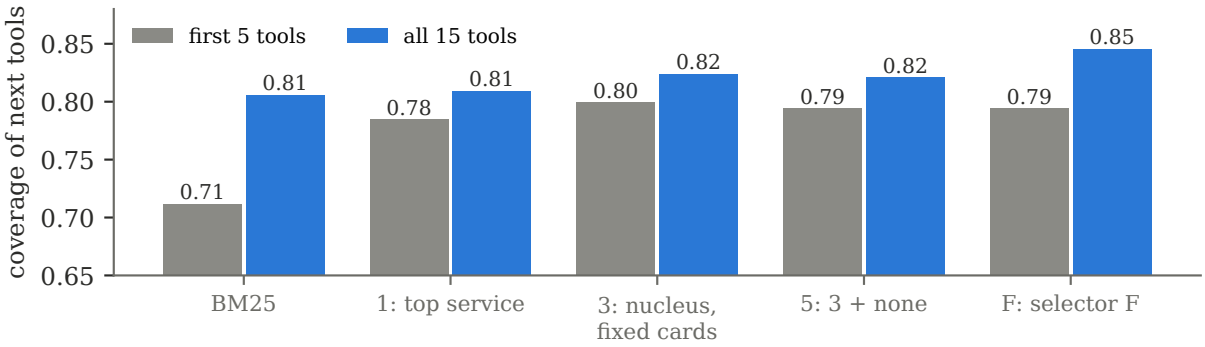


Figure 3: Held-out check on 500 agent searches from a different batch: share of the next tools covered by the first 5 and by all 15 returned tools. Variant numbers as in Table 3. The vertical axis starts at 0.65.

times as much as variant 3, at about nine DM calls per search; yes/no over all tools needed five calls and cost about 18 times as much. Service-based variants needed about 2.1–2.2 calls per search.

Selector F combines the winners and was checked on held-out searches. Selector F is variant 3 with the “none” option of variant 5, the named-service rule of variant 6, the “expert first” question for prerequisites as names, and usage lists; its first five tools come from the same question as variants 3–6, so its hit@1 on hand-written queries is 0.92. On 500 held-out agent searches from a different batch, selector F covered the most next tools within 15, 0.845, with all next tools covered for 79% of searches, against 0.806 and 75% for BM25; within the first five it covered 0.79 against 0.71, and it ranked the named service first for 99.7% of named queries against 85% for BM25 (Figure 3). It needs about three DM calls per search.

Table 4: Offline retrieval on 120 hand-written queries. Hit@k: a right tool is among the first k. Cost per 1,000 queries and median latency of the selecting call at 613 tools; name, BM25 and embedding search run locally. Bold: best per accuracy column.

	613 tools		1,076 tools			
Mechanism	hit@1	hit@5	hit@1	hit@5	$/1k queries	latency, s
Claude Code style	0.48	0.71	0.45	0.65	0	0.01
BM25	0.54	0.75	0.49	0.76	0	<0.01
embeddings	0.51	0.80	0.47	0.75	≈0	0.67
LLM router	0.86	**0.98**	0.82	**0.98**	0.44	1.31
DM hierarchy	**0.93**	0.97	**0.88**	0.97	0.15	0.54

6.6 The DM against shipped mechanisms and an LLM router

On the 120 hand-written queries, a simple server-then-tool DM hierarchy ranked better than every shipped mechanism and an LLM router that reads one line per tool (Table 4). At 613 tools it put the right tool first for 93% of queries, against 48–54% for the Claude Code-style, BM25 and embedding mechanisms and 86% for the router, which cost three times as much per query and took 2.4 times as long. Adding 463 decoy-rich tools lowered hit@1 by 3–6 points for every mechanism and left the order unchanged. General-purpose embeddings did poorly, as ToolRet found [28]. On 183 queries written by agents in live runs the gap was smaller: hit@1 0.60 for the DM against 0.45 for BM25, and hit@5 0.70 against 0.66, because agents write keyword-style queries that suit lexical search.

6.7 External benchmark: ToolRet

To compare the DM with published tool retrievers, we ran the full ToolRet benchmark [28]: 7,961 queries over 44,453 tools drawn from 35 datasets, with the original documentation, the paper's metric NDCG@10, and its average, the mean of the web, code and customized categories, each the mean of its tasks. ToolRet's tools have no services, so the DM re-ranks candidates instead of choosing services first: the union of the top 50 tools from BM25 and from text-embedding-3-small, 100 in all. Method A asks one multiple-choice question over the 100 candidates with short cards. Method E shows each tool's documentation, up to 1,500 characters, in four parallel questions of 25 tools and a final question over the six best of each. Both ask "A retrieval system is ranking tools for a user's task. Which tool is most relevant to it?". ToolRet reports two settings: with an instruction, a sentence of about 20 words that GPT-4o wrote for each query with the target tools in view, and without it, the query alone. The methods were fixed on a 385-query pilot and the analysis plan was written before the run. Our BM25 reproduced the paper's, 36.37 against 36.46 with the instruction and 22.23 against 22.32 without, and no DM call failed.

The DM was best in both settings (Table 5). With the instruction, method E scored 53.6, 2.3 points above the best published system, a re-ranker and a retriever both trained for tool retrieval, and 6.0 above the best re-ranker in ToolRet. Without the instruction, the setting closest to an agent's own query, method A scored 41.0 against 35.5, and its gain over BM25 grew from 15 to 19 points. On the 7,576 queries outside the pilot the DM scored 53.8 and 51.9 with the instruction and 41.6 without, so choosing the methods on the pilot did not inflate the result. Full documentation helped most where tools differ in detail: E beat A by 3.4 points on customized apps and by 1.0 on code. The DM fee for all 7,961 queries was $7.73 for both methods with the instruction and $1.16 for A without it. Two differences from the published systems remain: the DM re-ranks our own

Table 5: ToolRet, all 7,961 queries: NDCG@10 averaged over the three categories as in ToolRet. Published rows are quoted from ToolRet [28] and, for Tool-Embed, Tool-Rank and Qwen3, from Tool-REX [15], which reports only the setting with instruction. The DM rows re-rank the union of BM25 and embedding candidates; the published re-rankers re-rank the top 100 of NV-Embed-v1 or Tool-Embed-4B.

System	with instruction	without instruction
DM, method E	**53.6**	–
DM, method A	51.5	**41.0**
Tool-Embed-4B + Tool-Rank-4B, trained on ToolRet's documents	51.2	–
Tool-Embed-4B, trained	49.2	–
bge-reranker-v2-gemma	47.5	35.5
jina-reranker-v2-base	47.2	33.6
Qwen3-Embedding-8B	46.2	–
NV-Embed-v1	42.7	33.8
gpt-3.5-turbo-1106 as re-ranker	38.8	30.8
text-embedding-3-small, our first stage	38.1	24.7
BM25	36.4	22.2

first stage rather than NV-Embed-v1 or Tool-Embed-4B, and it was not trained for the task.

7 Discussion

Fewer searches matter more than cheaper searches. A search costs the agent a full turn: the model re-reads its whole context to write the query and again to read the answer. Selector F cut searches by 32% with DeepSeek and 60% with Sol; the tokens saved came from fewer LLM calls, not from shorter answers, since selector F's answers are longer than BM25's. A better first answer, with the right service's likely next tools listed by name, saves the turns that a rephrased query would cost.

When the DM pays off. The fee is fixed per search, while the saving is a share of the agent's own cost. The DM therefore pays off with expensive agent models, many tools and search-heavy work, and does not with inexpensive models such as DeepSeek V4.1 Flash, whose whole run costs about a cent. An efficient agent that searches little, like Sol, gains fewer searches to remove, and where cache writes dominate cost, as with Sol, fewer cached reads save little.

What "universal" means here. Selector F needs one API call per question, no training, and only the query and the catalog; it worked unchanged with two agent models and slots into any harness that has a search tool. We have not shown that it helps in every harness, catalog or kind of task.

What the DM is good and bad at. The DM is good at comparing options: choosing a service among cards or a tool among a service's tools. It is poor at judging a tool in isolation, as the yes/no variants showed. Its answers depend on the option texts: the Git-family errors came from service cards, not from the model.

8 Threats to validity

Scope. The services are simulated: real services return longer and messier results, which would raise token counts and could change how agents search. The tasks are coding tasks only. Two agent models were tested, and the agent runs used one harness, Hermes Agent; opencode results come from an earlier selector and task set.

Design choices made before the tests. Selector F was designed in exploratory runs on a separate long task and in the offline study, before the agent runs; we checked it on held-out searches from a different batch. The usage lists come from the same task suite but are built leave-one-task-out. The connected-services note in every prompt may raise search rates above those of an unprompted agent; it is the same for both conditions.

Exclusions. `k8s_crashloop` was excluded for both conditions before the runs; no run was dropped for its result.

Drift and baselines. Our A/A comparison could not rule out drift of the size we measure, so only same-time pairs support the claims. The baseline is Hermes Agent's BM25 as shipped; a tuned BM25 or query rewriting might narrow the gap.

9 Related work

Tool retrieval for large catalogs. Retrieving tools before generation is the standard answer to large catalogs [7, 26]. AnyTool [9] selects among over 16,000 APIs with a hierarchy of LLM agents, category first and tool second, and MCP-Zero [10] brings the server-then-tool idea to MCP; selector F applies the same hierarchy with a decision model instead of an LLM. RAG-MCP [11] retrieves MCP tool descriptions with embeddings to reduce prompt bloat, and Mudunuri et al. [20] select among 121 tools of five MCP servers with embeddings, without agent runs. ToolRet finds that general-purpose embedding retrievers do poorly on tool retrieval [28]; Tool-REX expands ToolRet's documents with an LLM and trains a retriever and a re-ranker on them [15], and Lee [13] find that selecting a tool is largely retrieval. Agent skills [1] apply progressive disclosure to instructions; our service cards and handbook play this role for the catalog.

MCP benchmarks. LiveMCPBench [18], MCPMark [31], MCP-Atlas [5] and Toolathlon [14] score agents' task completion with real MCP tools. We fix the agent, vary only the selector, and simulate back ends so that runs are reproducible and writes can be graded safely.

Vendor reports. Stacklok compared its MCP Optimizer with Anthropic's tool search on 2,792 tools [29], and Arcade tested Anthropic's tool search on about 4,000 tools [12]. Both measure selection accuracy, not task completion or tokens per task.

Decision models. The Artificial Analysis Decision Index [4] and an analysis of it [22] compare DMs with LLMs on standalone decisions, including tool calling. We place a DM inside agents as the tool selector and measure its effect on whole runs.

10 Conclusion

A decision model can serve as an agent's tool selector. Built on Microsoft Decision 1, selector F chooses services with a "none" option, then asks two questions per service and adds usage-based names, at about three DM calls per search. Offline it put the right tool first for 92% of hand-written queries at 1,076 tools, against 49% for BM25, and on ToolRet it re-ranked candidates better than every published retriever and re-ranker. Inside Hermes Agent, in pre-planned same-time paired runs, it used 9.7% fewer tokens than BM25 with GPT-6.1 Sol and 8.5% fewer with DeepSeek V4.1 Flash, with 32–60% fewer searches and equal pass rates. Its fee left Sol's cost per run unchanged and made DeepSeek runs 25% more expensive. These results hold for coding agents with simulated services, two agent models and one harness; testing other harnesses, real services and non-coding work is the next step.

Reproducibility. We will release the tool harvester, catalog snapshots, simulator, tasks with hidden tests and reference solutions, the selector and ranking server, the Hermes patch, the sandbox setup, all run records, the analysis plans and the analysis and figure scripts.

References

[1] Anthropic. Agent skills overview (Claude platform documentation). https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview, 2025. Accessed 6 November 2026.

[2] Anthropic. Tool search tool (Claude platform documentation; regex and BM25 variants). https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool, 2025. Accessed 6 November 2026.

[3] Anthropic. Claude code documentation: Scale with MCP tool search. https://code.claude.com/docs/en/mcp, 2026. Accessed 6 November 2026.

[4] Artificial Analysis. Artificial analysis decision index. https://artificialanalysis.ai/evaluations/artificial-analysis-decision-index, 2026. Accessed 6 November 2026.

[5] Chaithanya Bandi et al. MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers. *arXiv preprint arXiv:2602.00933*, 2026.

[6] Block. goose: Tool selection strategy (vector and LLM-based tool routers, preview feature). https://web.archive.org/web/20251031035432/https://block.github.io/goose/docs/guides/managing-tools/tool-router/, 2025. Preview in 2025. The vector strategy was removed in https://github.com/block/goose/pull/3933 (August 2025), the LLM-based strategy and its documentation in https://github.com/block/goose/pull/6250 (December 2025).

[7] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference. *arXiv preprint arXiv:2603.13426*, 2026.

[8] DeepSeek. DeepSeek V4.1 Flash. https://openrouter.ai/deepseek/deepseek-v4.1-flash-1017, 2026. OpenRouter model page; model identifier `deepseek/deepseek-v4.1-flash-1017`.

[9] Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 11812–11829, 2024. arXiv:2402.04253.

[10] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero: Active tool discovery for autonomous LLM agents. *arXiv preprint arXiv:2506.01056*, 2025.

[11] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[12] Eric Gustin. We threw 4,000 tools at Anthropic's new tool search. Here's what happened. https://www.arcade.dev/blog/anthropic-tool-search-4000-tools-test/, December 2025. Arcade.dev blog, 3 December 2025.

[13] Janghoon Lee. Selection is retrieval, abstention is not: On-device tool routing over 70 Korean-English actions. *arXiv preprint arXiv:2609.18672*, 2026.

[14] Junlong Li et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.25726.

[15] Xuan Lu, Haohang Huang, Rui Meng, Yaohui Jin, Wenjun Zeng, and Xiaoyu Shen. Tools are under-documented: Simple document expansion boosts tool retrieval. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.22670.

[16] Microsoft. Use tools with agents (Visual Studio Code documentation: 128-tool limit and virtual tools). https://code.visualstudio.com/docs/copilot/agents/agent-tools, 2026. Accessed 6 November 2026.

[17] Microsoft. Not every decision needs an LLM: Getting started with Microsoft-Decision-1. https://techcommunity.microsoft.com/blog/educatordeveloperblog/not-every-decision-needs-an-llm-getting-started-with-microsoft-decision-1/4563468, October 2026. Microsoft Community Hub, Educator Developer Blog.

[18] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Qianhao Yuan, Xuanang Chen, Yaojie Lu, et al. LiveMCPBench: Can agents navigate an ocean of MCP tools? *arXiv preprint arXiv:2508.01780*, 2025.

[19] Model Context Protocol. Model context protocol specification. https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[20] Sarat Mudunuri, Jian Wan, Ally Qin, and Srinivasan Manoharan. Semantic tool discovery for large language models: A vector-based approach to MCP tool selection. *arXiv preprint arXiv:2603.20313*, 2026.

[21] Nous Research. Hermes agent. https://github.com/NousResearch/hermes-agent, version 0.19.0 (PyPI), 2026.

[22] Randal S. Olson. When to use a decision model instead of an LLM. Blog post, 15 October 2026, https://www.randalolson.com/2026/10/14/when-to-use-a-decision-model-instead-of-an-llm/, 2026.

[23] OpenAI. Codex CLI. https://github.com/openai/codex, 2026. MCP tool search with BM25 in `codex-rs/core/src/tools/handlers/tool_search.rs`; source checked 6 November 2026.

[24] opencode contributors. opencode: an open-source coding agent. https://opencode.ai, version 1.19.4, 2026.

[25] OpenRouter. Decisions API reference. https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request, 2026. Accessed 6 November 2026.

[26] Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

[27] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[28] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24497–24524, 2025. arXiv:2503.01763.

[29] Stacklok. Stacklok's MCP optimizer vs Anthropic's tool search tool: A head-to-head comparison. https://stacklok.com/blog/stackloks-mcp-optimizer-vs-anthropics-tool-search-tool-a-head-to-head-comparison/, 2025.

[30] TypeSafe AI. Jev 1.14 decision model. https://typesafe.ai; served via OpenRouter as `typesafe/jev-1.14`, https://openrouter.ai/typesafe/jev-1.14, 2026.

[31] Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, et al. MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use. *arXiv preprint arXiv:2509.24002*, 2025.

A Sandbox hardening

The isolation of §4 is the result of fixes made during development; runs made before them are not reported. In order:

- Agents run as an unprivileged user with an empty environment and no credentials; experiment files are readable only by root and the API key is held by a local proxy that admits only the experiment models, because harness-level permissions alone let an agent read hidden tests through absolute paths.
- Each Hermes run gets its own mount namespace with a private work directory, home directory and `/tmp`, so runs cannot see each other's files or leftovers.
- The MCP server accepts only requests that carry a per-run token the agent's shell cannot read, so the shell cannot list the catalog or query the ranker directly.
- Hermes's deny list blocks probing commands: `curl`, `wget`, `ps`, `ss`, `netstat`, `nmap`, `lsof`, `/proc`, `localhost` and `find /`.
- Rankers are compared inside Hermes's own bridge rather than through a separate search tool with a generic call tool, so that only the ranking differs.

- The provider and sampling are pinned as described in §4.

B Tool catalog and task set

Table 6: The real tool catalog: tools per MCP server, from each server's own `tools/list`.

Catalog	Servers with their numbers of tools	Tools
main	Linear 198, Atlassian 98, GitHub 91, Grafana 81, Git 28, MongoDB 27, Playwright 25, Notion 24, Kubernetes 23, Sentry 9, Slack 8, Postgres 1	613
extra	GitLab 127, monday 84, Salesforce 74, Heroku 33, HubSpot 21, AWS CloudWatch 19, AWS EKS 16, Airtable 16, Filesystem 14, CircleCI 13, Browserbase 9, Memory 9, Netlify 9, Terraform 9, Elasticsearch 4, ClickHouse 3, Figma 2, Docker 1	+463
all	30 servers	1,076

Table 7: Task set v2, 38 tasks. Prompts are shortened to their first sentence and shown without the escape clause. Services: the servers of the needed tools, in order; $+w$: number of required writes; $\star$: new in v2.

Category	Task	First sentence of prompt	Services
control	`c_chunk`	chunk in batching/chunk.py drops the last partial chunk.	–
control	`c_dedupe`	dedupe in collections_util/dedupe.py loses the original order.	–
control	`c_duration`	parse_duration in timeutil/duration.py only understands hours.	–
control	`c_leap`	is_leap in calendar_util/leap.py is wrong for some years.	–
single	`confluence_limits`	Our public API rate limits changed.	atlassian
single	`eks_importer` $\star$	The data team's nightly batch-importer job has failed three nights in a row.	aws_eks
single	`gh_ci_fail`	Pull request #214 in acme/payments can't be merged because a check is red.	github $+1w$
single	`gh_codesearch`	SUPPORTED_CURRENCIES in shop/currencies.py has drifted from the company-wide canonical list, which lives in another repository.	github
single	`gh_dependabot`	Security says acme/payments has dependencies with known vulnerabilities.	github
single	`gh_issue`	Please implement acme/payments#301.	github
single	`gh_open_pr`	Pagination in acme/catalog is broken (see #57).	github $+2w$
single	`gh_release`	We are behind on the ledger API.	github
single	`gh_review`	There is reviewer feedback waiting on pull request #88 in acme/ledger.	github $+1w$
single	`grafana_incident`	We are in the middle of an incident about checkout timeouts and the responders agreed on a mitigation.	grafana $+1w$
single	`jira_impl`	Pick up PAY-412: implement it, then move it to review.	atlassian $+1w$

Category	Task	First sentence of prompt	Services
single	`k8s_crashloop`	ledger-worker keeps restarting in production (namespace ledger) since the last config change.	kubernetes
single	`k8s_oom`	report-builder (namespace reports) keeps dying in production.	kubernetes
single	`linear_bug`	Fix ENG-77 and move it to In Review.	linear +1w
single	`loki_errors`	Customers stopped getting shipping e-mails at around 06:00 UTC today; the notifier service is the suspect.	grafana
single	`mongo_shape`	The nightly export has started crashing on recent orders.	mongodb
single	`notion_runbook`	We are rotating the webhook signing keys.	notion
single	`oncall_contact`	Set ESCALATION_CONTACT in ops/escalation.py to the e-mail address of whoever is on call for the payments team right now.	grafana
single	`pg_migration`	Add migrations/0042_payments_merchant_idx.sql with an index named payments_merchant_created_idx for looking up a merchant's payments by creation time.	postgres
single	`prom_threshold`	Set SLOW_REQUEST_MS in alerts/config.py to twice the checkout service's current p99 request latency, in milliseconds.	grafana
single	`sentry_events`	Since this morning's deploy some clients get HTTP 500 from POST /refunds.	sentry
single	`sentry_fix`	The noisiest production error of payments-api right now needs to go.	sentry +1w
single	`slack_decision`	The payments team agreed on a new webhook retry policy in a chat discussion.	slack +1w
multi	`cw_reports` ⋆	The legacy reports API has been answering 502 since yesterday evening.	aws_cloudwatch, slack +1w
multi	`gl_pipeline` ⋆	The main-branch pipeline of billing-legacy is red.	gitlab +1w
multi	`linear_notion_pr` ⋆	Do ENG-91 the way our API guidelines say, and open a pull request for it from a branch named eng-91 against main in acme/public-api.	linear, notion, github +1w
multi	`multi_alert`	Something is paging for the checkout service.	grafana, slack +1w
multi	`multi_jira_conf`	Implement PAY-530 as its design document specifies, then leave a one-line summary on the ticket.	atlassian +1w
multi	`multi_sentry_gh`	A recent change to ledger-worker introduced a reconciliation bug that shows up as a production error.	sentry, github +1w
multi	`promo_sentry_jira` ⋆	Android users report they can't apply promo codes.	sentry, atlassian +1w
absent	`absent_jira`	Implement PAY-999 in payments/limits.py.	–
absent	`absent_release`	Pin SDK_VERSION in sdk/version.py to the newest release of acme/ghost-sdk (without the leading v).	–

Category	Task	First sentence of prompt	Services
absent	`absent_sentry`	The billing-cron error someone mentioned in standup needs fixing in billing/cron.py.	–
absent	`absent_slack`	Set CACHE_TTL_S in cache/config.py to the value the platform team agreed on.	–